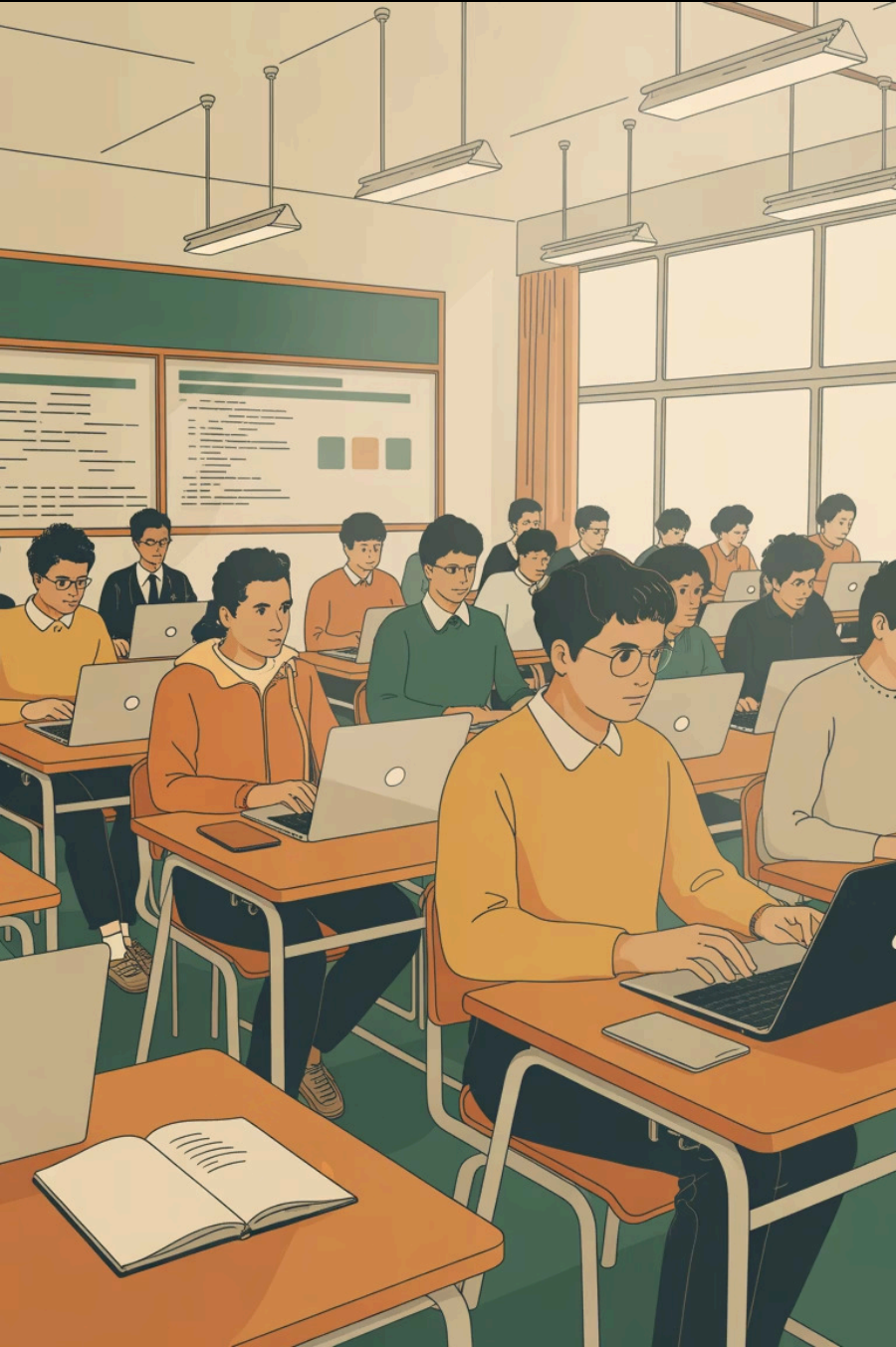




ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Frameworks Back-end, MVC e Arquitetura em Camadas

Estruturas, padrões e boas práticas para o desenvolvimento de aplicações modernas

Aula de hoje

Objetivos da Aula

1

Frameworks Back-end

Compreender o conceito de framework e conhecer os principais exemplos por linguagem

2

Padrão MVC

Entender o padrão Model-View-Controller e as responsabilidades de cada camada

3

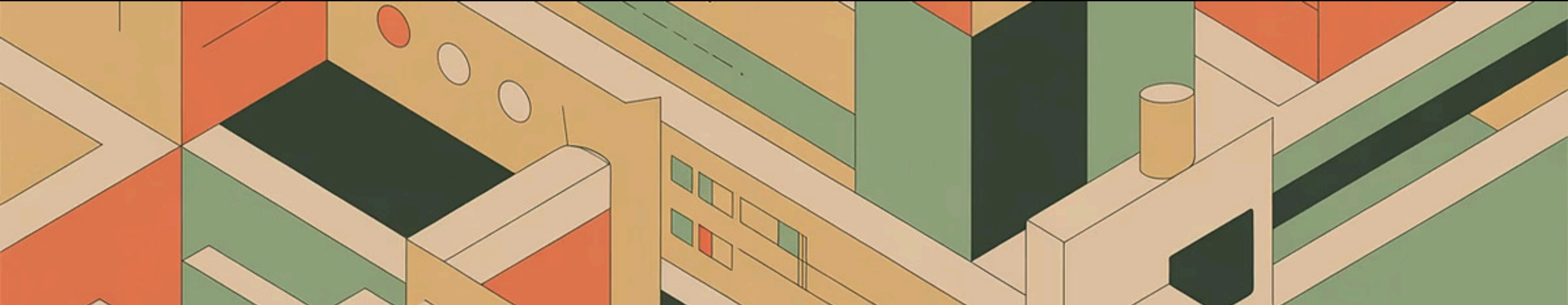
Arquitetura em Camadas

Conhecer a organização em Apresentação, Aplicação, Domínio e Infraestrutura

4

Relações e Prática

Identificar como MVC se relaciona com arquitetura em camadas e aplicar em atividade prática



O que é um Framework Back-end?

Conceito

Um framework é uma **fundação pré-construída** que resolve problemas comuns, impõe uma arquitetura e inverte o controle do fluxo da aplicação.

O desenvolvedor **preenche as lacunas** dentro de uma estrutura já estabelecida.

Por que usar?

- **Produtividade:** reutilização de código testado
- **Segurança:** proteções contra SQL Injection, XSS, CSRF
- **Padrões:** arquitetura organizada desde o início
- **Comunidade:** documentação e suporte amplos

Principais Frameworks por Linguagem



Java

Spring Boot — Escalável, enterprise

Jakarta EE — Padrão corporativo

Quarkus — Cloud-native, leve



Node.js

Express — Minimalista, flexível

NestJS — Estruturado, TypeScript

Fastify — Alta performance



Python

Django — Baterias incluídas, ORM

FastAPI — Moderno, async, rápido



PHP

Laravel — Sintaxe expressiva, ecossistema maduro

Java - Frameworks Back-end



Spring Boot

Framework **líder no ecossistema Java** para construção de aplicações robustas e escaláveis, simplificando o desenvolvimento com **configuração mínima** e um modelo "convention over configuration".

Uso: Mais de 60% dos desenvolvedores Java utilizam Spring Boot, com milhões de downloads diários de artefatos.

spring.io/spring-boot



JAKARTA EE

Jakarta EE (antigo Java EE)

Conjunto de **especificações e APIs** para desenvolver aplicações corporativas distribuídas e multicamadas. Foco em **padrões e portabilidade** entre diferentes servidores de aplicação.

Uso: Milhares de aplicações corporativas e governamentais são construídas com Jakarta EE, com uma comunidade ativa e grandes empresas adotando suas especificações.

jakarta.ee

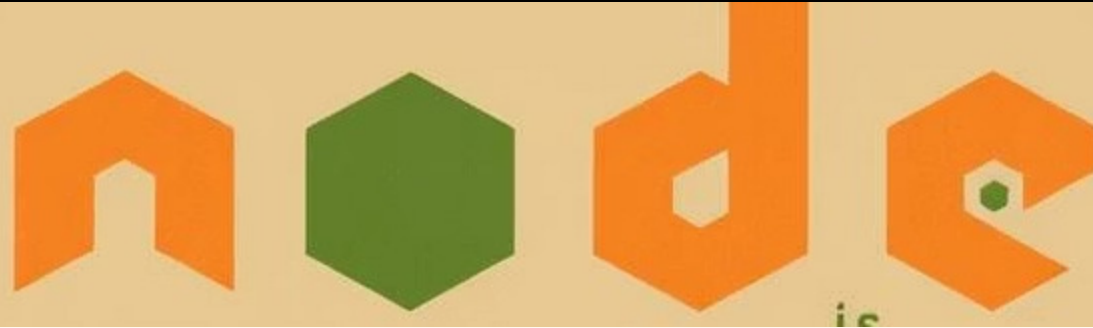


Quarkus

Um **framework Java nativo para a nuvem** otimizado para **GraalVM e OpenJDK HotSpot**, projetado para inicialização rápida e baixo consumo de memória, ideal para microsserviços e funções serverless.

Uso: Ganhou significativa tração nos últimos anos, com mais de 10 milhões de downloads de suas extensões e uma crescente adoção em ambientes cloud-native.

quarkus.io



Node.js - Frameworks Back-end



Express

Um framework web minimalista e flexível para Node.js que fornece um conjunto robusto de recursos para aplicações web e APIs. Permite grande liberdade na escolha de bibliotecas e padrões.

Uso: Um dos frameworks mais populares do Node.js, com milhões de downloads semanais e uma vasta comunidade de usuários.

expressjs.com



NestJS

Um framework progressivo do Node.js para a construção de aplicações server-side eficientes, confiáveis e escaláveis. Utiliza TypeScript e combina elementos de programação orientada a objetos (POO), programação funcional e programação reativa (RxJS).

Uso: Crescimento rápido, com centenas de milhares de downloads semanais e adoção crescente em grandes empresas e startups.

nestjs.com



Fastify

Um framework web para Node.js altamente focado em oferecer a melhor experiência de desenvolvimento com um mínimo de sobrecarga de desempenho e um sistema de plugins robusto.

Uso: Reconhecido por sua alta performance, com dezenas de milhares de downloads semanais e ideal para microsserviços de alta demanda.

fastify.io

Python - Frameworks Back-end



Django

Um framework web de alto nível que incentiva o desenvolvimento rápido e um design limpo e pragmático. Inclui um ORM (Object-Relational Mapper), sistema de administração e autenticação, oferecendo uma abordagem "baterias incluídas" para aplicações robustas.

Uso: Um dos frameworks Python mais maduros, amplamente utilizado para construir complexos sistemas web, com centenas de milhares de sites ativos e milhões de downloads mensais.

djangoproject.com



FastAPI

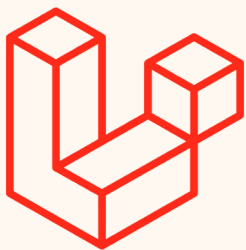
Um framework web moderno e rápido (de alto desempenho) para construir APIs com Python 3.7+ baseado em type hints padrão do Python. Ideal para criar APIs RESTful, ele é assíncrono por design e otimizado para velocidade.

Uso: Com um crescimento exponencial, tem dezenas de milhões de downloads mensais e é uma escolha popular para microsserviços e desenvolvimento de APIs de alta demanda.

fastapi.tiangolo.com



PHP - Frameworks Back-end



Laravel

Um dos frameworks PHP mais populares e completos, conhecido por sua sintaxe elegante, ferramentas robustas e vasta coleção de pacotes que aceleram o desenvolvimento de aplicações web.

Uso: Com uma vasta comunidade e milhões de desenvolvedores, Laravel é o framework PHP mais utilizado, com mais de 100 milhões de downloads de pacotes a cada mês.

laravel.com

Critérios de Comparação entre Frameworks

Como escolher o framework mais adequado para cada projeto

Performance

Latência, throughput e consumo de recursos sob carga

Curva de Aprendizado

Facilidade de adoção pela equipe e disponibilidade de documentação

Ecossistema

Bibliotecas, plugins, comunidade ativa e suporte a longo prazo

Escalabilidade

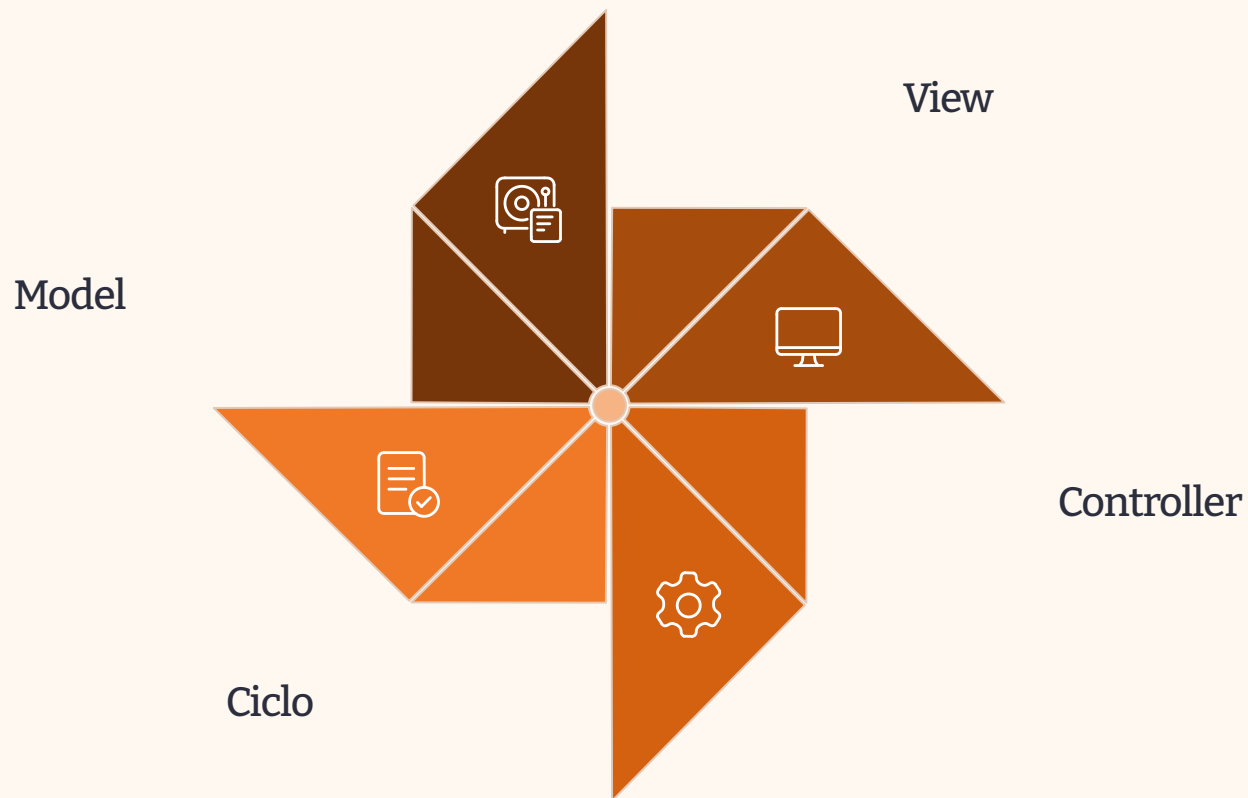
Capacidade de crescer junto com a demanda e o tamanho do projeto

Mercado de Trabalho

Demanda por profissionais e relevância nas vagas disponíveis

O Padrão MVC

Model — View — Controller



O MVC **separa responsabilidades** da aplicação em três componentes distintos, tornando o código mais organizado, testável e fácil de manter.

Responsabilidades no MVC



Model

- Representa os **dados** da aplicação
- Contém as **regras de negócio**
- Acessa e persiste dados (banco de dados)
- Independente de View e Controller



View

- Responsável pela **interface com o usuário**
- Apresenta os dados recebidos do Model
- Não contém lógica de negócio
- Templates HTML, JSON, XML etc.

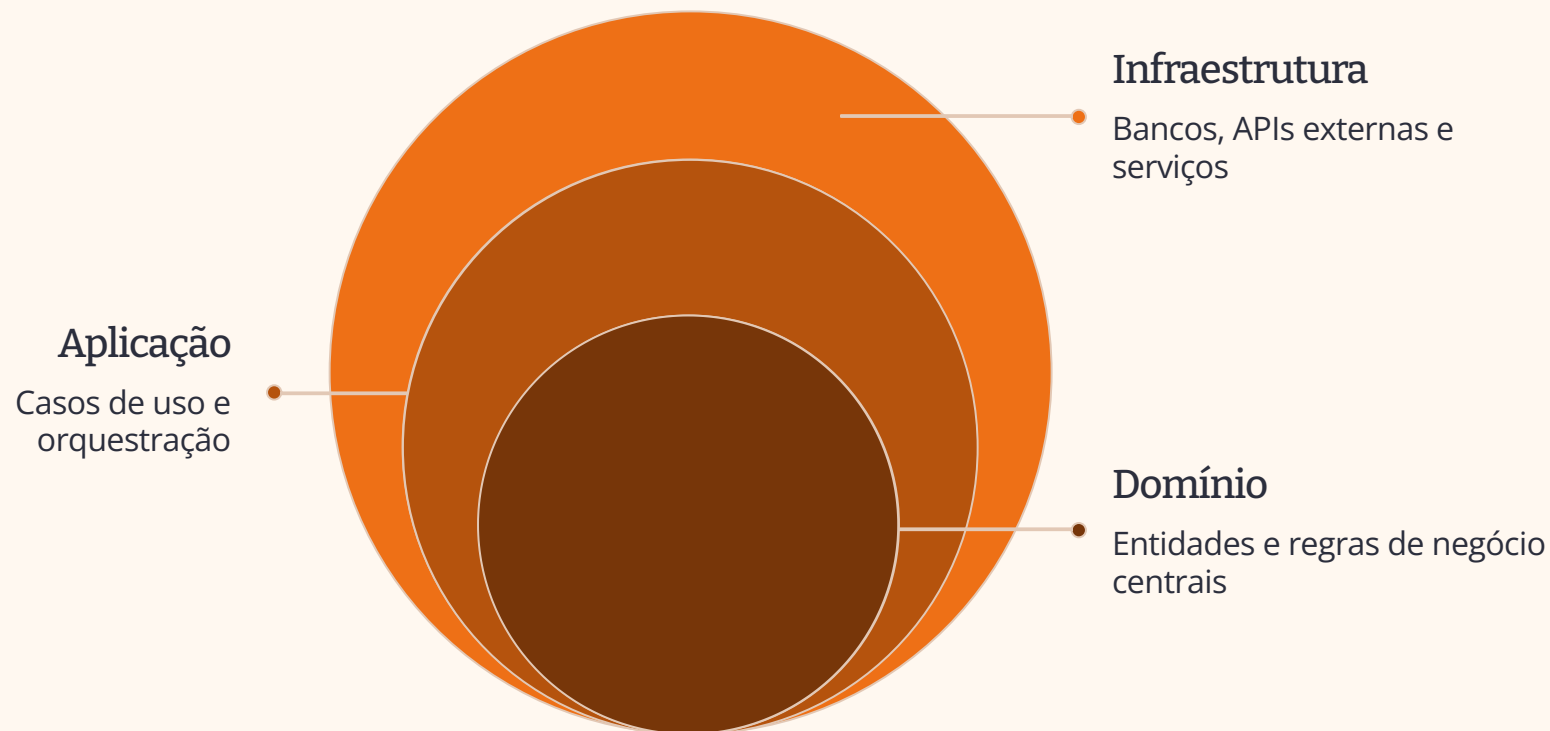


Controller

- Recebe as **requisições do usuário**
- Coordena Model e View
- Processa entrada e decide a resposta
- Não armazena dados nem renderiza HTML

Arquitetura em Camadas

Organização vertical de responsabilidades



Cada camada **se comunica apenas com a camada adjacente**, garantindo baixo acoplamento e alta coesão entre as responsabilidades do sistema.

MVC e Arquitetura em Camadas

Relação entre os conceitos

O MVC atua principalmente nas camadas de **Apresentação e Aplicação**.

- **View** → Camada de Apresentação
- **Controller** → Camada de Aplicação
- **Model** → Camadas de Domínio e Infraestrutura

São complementares: a Arquitetura em Camadas organiza o sistema inteiro, enquanto o MVC organiza a interação com o usuário.

Atividade Prática

Em grupos de 3 alunos:

1. Escolha um framework da aula (Spring Boot, NestJS, Django ou Laravel)
2. Identifique onde o padrão MVC aparece na estrutura de pastas do framework
3. Mapeie cada pasta/arquivo para as camadas da Arquitetura em Camadas
4. Apresente o mapeamento para a turma em **5 minutos**

Debate e Encerramento

Perguntas para Debate

Um framework impede o desenvolvedor de entender os fundamentos da programação?

Quando vale a pena abrir mão do MVC por outra arquitetura?

O que é mais importante: dominar um único framework ou conhecer vários superficialmente?

Resumo da Aula

01

Framework Back-end

Fundação reutilizável com inversão de controle

02

Exemplos por Linguagem

Java, Node.js, Python e PHP com seus frameworks

03

Padrão MVC

Model, View e Controller com responsabilidades separadas

04

Arquitetura em Camadas

Apresentação → Aplicação → Domínio → Infraestrutura

OBRIGADO PELA ATENÇÃO! 🏠